Answer to the Question no: 02

Sanity testing and regression testing are both important testing methodologies in software development, but they serve different purposes and are used in different contexts. Here are the key differences between the two:

Sanity Testing

1. **Purpose**:
   - Sanity testing is a quick, surface-level test to ensure that a particular function or bug fix works as expected.
   - It is performed to verify that the basic functionality of a specific part of the application is working correctly.
2. **Scope**:
   - Limited in scope; focuses only on specific areas of functionality that have been changed or added.
   - It does not involve a comprehensive test of the entire application.
3. **Timing**:
   - Usually conducted after receiving a software build to check for major issues before proceeding to more detailed testing.
   - It is often performed after a minor change or bug fix.
4. **Depth**:
   - Shallow testing, focusing on checking that the specific function or bug fix works correctly.
   - It does not cover extensive scenarios or edge cases.
5. **Objective**:
   - To ensure that the recent changes or fixes have not broken any primary functionality of the application.
   - To quickly determine if the build is stable enough for further testing.

Regression Testing

1. **Purpose**:
   - Regression testing aims to ensure that recent code changes have not adversely affected existing functionality.
   - It verifies that the software continues to work as expected after modifications.
2. **Scope**:
   - Broad in scope; includes testing the entire application to check for unintended side effects of recent changes.
   - It covers both the new changes and the existing functionalities.
3. **Timing**:
   - Performed after any code changes, such as new features, bug fixes, or improvements.

    - Often conducted repeatedly throughout the software development lifecycle.
4. **Depth**:
    - Comprehensive testing, including a wide range of scenarios and test cases.
    - It includes re-running previously executed test cases to ensure consistency.
5. **Objective**:
    - To identify any regressions or new bugs introduced by recent changes.
    - To maintain the overall quality and stability of the software over time.

Answer the Question no: 03

Exploratory Testing

- **Purpose**: To discover defects and issues by exploring the application without predefined test cases.
- **Approach**: Testers dynamically design and execute tests based on their knowledge, experience, and intuition.
- **Focus**: Identifying unexpected behavior and potential areas of concern.
- **Characteristics**: Unscripted, adaptive, and relies on tester's creativity and analytical skills.

Happy Path Testing

- **Purpose**: To verify that the application works as expected when used in a standard, ideal scenario.
- **Approach**: Testing the application using inputs and conditions that are expected to produce a successful result.
- **Focus**: Ensuring the primary functionality works correctly under normal conditions.
- **Characteristics**: Simple, straightforward, and does not cover edge cases or error handling.

Unit Testing

- **Purpose**: To validate the correctness of individual components or units of code (e.g., functions, methods).
- **Approach**: Isolating each unit and testing it independently from the rest of the application.
- **Focus**: Ensuring each unit performs as intended and meets its specifications.

- **Characteristics**: Automated, performed by developers, and often part of the continuous integration process.

Functional Testing

- **Purpose**: To verify that the application performs its intended functions correctly.
- **Approach**: Testing the application against functional requirements and specifications.
- **Focus**: Ensuring that each feature of the application behaves as expected.
- **Characteristics**: Can be manual or automated, black-box testing methodology, and covers a wide range of scenarios and inputs.

Answer to the Question no: 04

Prerequisites:

1. Python installed on our machine.
2. Then the Selenium Web Driver installed. We can install it using pip
3. ”pip install selenium”

Here's a sample script that automates the login and registration processes for a hypothetical QA Brain application.

1. ***Import necessary modules:***

main.py

```
1  from selenium import webdriver
2  from selenium.webdriver.common.by import By
3  from selenium.webdriver.common.keys import Keys
4  from selenium.webdriver.support.ui import WebDriverWait
5  from selenium.webdriver.support import expected_conditions as EC
6  import time
7
```

2. Initialize Web Driver

```
9
10 driver = webdriver.Chrome(executable_path='path/to/chromedriver')  # Update with your chromedriver path
11 driver.maximize_window()
12 driver.get('http://qabrain.com')  # Update with the actual URL of QA Brain
13
```

4. Registration Process:

```
14 def register():
15     driver.find_element(By.LINK_TEXT, 'Register').click()  # Adjust based on the actual element
16     time.sleep(2)  # Adjust based on the actual element
17
18     driver.find_element(By.ID, 'username').send_keys('testuser')  # Adjust based on the actual element
19     driver.find_element(By.ID, 'email').send_keys('testuser@example.com')  # Adjust based on the actual element
20     driver.find_element(By.ID, 'password').send_keys('Password123')  # Adjust based on the actual element
21     driver.find_element(By.ID, 'confirm_password').send_keys('Password123')  # Adjust based on the actual element
22     driver.find_element(By.ID, 'register').click()  # Adjust based on the actual element
23
24     # Wait for registration confirmation
25     WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.ID, 'registration_success')))
26     print("Registration successful")
27
```

5. Login process

```
28     def login():
29     driver.find_element(By.LINK_TEXT, 'Login').click()  # Adjust based on the actual element
30     time.sleep(2)
31
32     driver.find_element(By.ID, 'username').send_keys('testuser')  # Adjust based on the actual element
33     driver.find_element(By.ID, 'password').send_keys('Password123')  # Adjust based on the actual element
34     driver.find_element(By.ID, 'login').click()  # Adjust based on the actual element
35
36     # Wait for login confirmation
37     WebDriverWait(driver, 10).until(EC.presence_of_element_located((By.ID, 'login_success')))
38     print("Login successful")
39
```

6. Execute the Functions:

```
40     try:
41     register()
42     login()
43 finally:
44     driver.quit()
45
```

Explanation:

1. **Initialization**: Set up the Web Driver and navigate to the QA Brain site.
2. **Registration**: Locate and interact with the registration form elements to fill in user details and submit the form.
3. **Login**: Locate and interact with the login form elements to log in with the registered user credentials.
4. **Execution**: Call the functions to perform registration and login, then quit the driver.